A CORE YOU CAN EXPLAIN

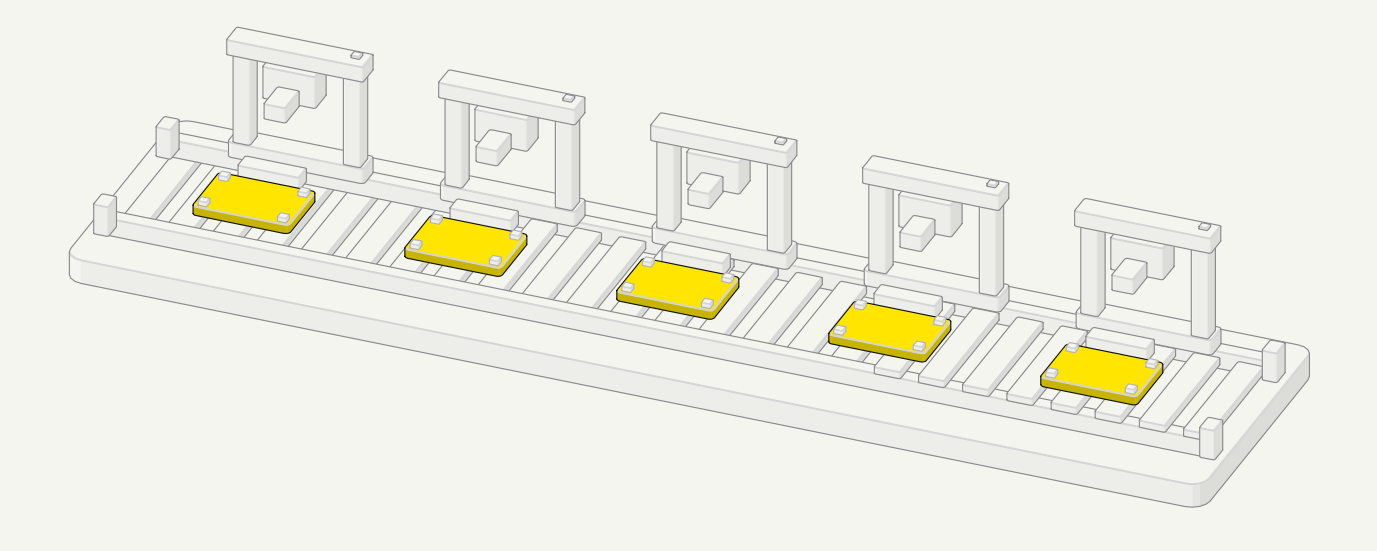
RISC-V, one clock edge at a time

A visual field textbook for building an educational five-stage RV32I core with a signed dot-product instruction.

Written for Rahil's October 17–18 build sprint. Start with bits. Finish with a checked instruction trace and a reproducible FPGA timing estimate.

A student trynna learn.

Worked examples · diagrams · labs · answer key



Two days, one coherent project

You do not need to memorize every opcode. You need a model that predicts what the hardware will do. Each chapter builds that model, then asks you to test it.

Today: understand the machine

Read pages 3–21. Run the reference, trace dependencies, then add two hazard tests and one DOT4 example. Allow 4–5 focused hours. The questions marked Q1–Q12 check the main ideas.

Tomorrow: earn the evidence

Read pages 22–29. Run both simulators, interpret coverage, route the measurement shell and complete one controlled experiment. Allow 5–7 hours. Use the worked answers after making your own prediction.

What already exists

The repository contains an assistant-generated reference implementation. There was no earlier draft. Your additions, explanations and experiment notes establish what you personally understand and changed.

Scope: an educational execution core with a specific memory contract. A full privileged processor, external certification and a physical board demonstration are follow-up projects.

Read	Build question	Evidence
Pages 3–10	What does one instruction mean?	Hand-calculated examples
Pages 11–19	What changes when instructions overlap?	Hazard and flush tests
Pages 20–24	What does DOT4 add?	Signed-lane and dependency checks
Pages 25–29	Which claims did the tools establish?	Coverage and route reports

A bit pattern needs an interpretation

A **bit** holds 0 or 1. A **byte** contains eight bits. A 32-bit word contains four bytes. Binary positions carry powers of two, with the least significant bit at position zero.

Hexadecimal uses one digit per four bits. This makes a 32-bit instruction readable as eight hex digits. The prefix `0x` means hexadecimal.

> **WORKED EXAMPLE**
>
> $0x2D = 0010\ 1101_2 = 32 + 8 + 4 + 1 = 45.$

Bitwise AND keeps positions where both inputs have a 1. OR keeps positions where either input has a 1. XOR keeps positions that differ. These operations act on each bit separately.

A **multiplexer**, or mux, selects one input value. Forwarding uses muxes to choose between register values and newer pipeline results.

FIG 01 / UNSIGNED BYTE

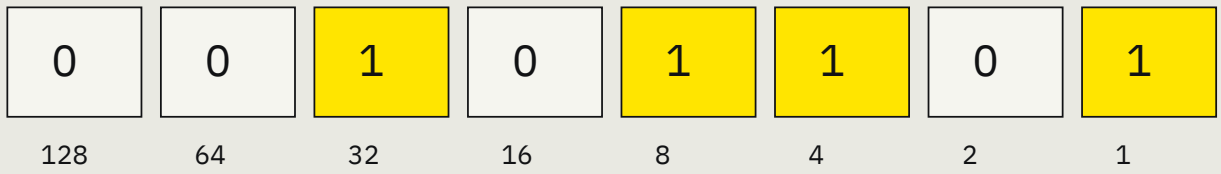


Illustration: each set bit contributes its place value.

> **CHECK YOUR MODEL**
>
> Q1. What decimal value does 0x80 represent as an unsigned byte? What is 0xA5 AND 0x0F?

Signed arithmetic reuses the same bits

An unsigned byte spans 0 through 255. A signed two's-complement byte spans −128 through 127. For an n-bit value with its top bit set, subtract 2^n from its unsigned value.

> **WORKED EXAMPLE**
>
> `0xFF` is 255 unsigned, but `255 - 256 = -1` signed. `0x80` is 128 unsigned, but −128 signed.

Sign extension copies the sign bit into new upper positions. Extending signed `0x80` to 32 bits gives `0xFFFFFF80`. Zero extension gives `0x00000080`.

A logical right shift inserts zeros. An arithmetic right shift copies the sign bit. RV32I uses SRL for the first and SRA for the second. Signed comparisons interpret the sign bit. Unsigned comparisons treat every bit as magnitude.

Integer addition wraps modulo 2^{32} in this core. `0xFFFFFFFF + 1` gives zero. Ordinary ADD does not raise an overflow trap.

Bits	Unsigned 8-bit	Signed 8-bit	32-bit sign extension
01111111	127	127	0x0000007F
10000000	128	−128	0xFFFFFF80
11111111	255	−1	0xFFFFFFFF

> **CHECK YOUR MODEL**
>
> Q2. Interpret 0xFE as signed 8-bit. Compare 0xFFFFFFFF with 1 using SLT and SLTU.

A clock edge separates old state from new state

Combinational logic computes outputs from current inputs. An adder, comparator or mux is combinational. A **register** stores bits across time. It updates at an active clock edge.

Between edges, signals travel through logic and wires. Before the next edge, the destination register needs a stable input. A path that takes too long limits clock frequency.

> **WORKED EXAMPLE**
>
> With a 20 ns clock period, the nominal frequency is `1 / 20 ns = 50 MHz`. The useful combinational budget is smaller because registers require setup time and clocks have uncertainty.

A pipeline puts registers between work regions. It overlaps different instructions across those regions. It does not make one addition logically need fewer steps.

The actual core uses a synchronous, active-high reset. Reset takes effect on a rising edge. It clears pipeline valid bits and sets the program counter to zero.

FIG 02 / REGISTER BOUNDARY

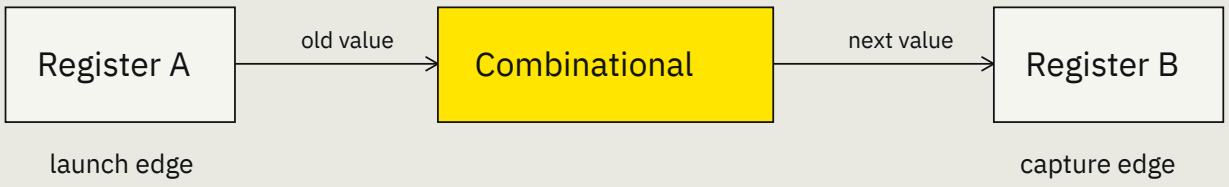


Illustration: combinational logic settles between active edges.

> **CHECK YOUR MODEL**
>
> Q3. A design needs at least 25 ns per clock period. What maximum frequency does that imply?

RTL describes concurrent hardware

Register-transfer level, or RTL, describes stored state and the logic between registers. Verilog source order is not a CPU instruction schedule. Separate hardware blocks operate concurrently.

```
always @(posedge clk) begin
    if (reset) begin a ⩽ 0; b ⩽ 0; end
    else begin a ⩽ input_value; b ⩽ a; end
end
```

Nonblocking assignments (⩽) schedule state changes after the block evaluates. Here, b receives the old a. That behavior creates a pipeline boundary.

```
always @* begin
    result = 0;
    case (select)
        0: result = a + b;
        1: result = a ^ b;
        default: result = 0;
    endcase
end
```

Blocking assignments (=) are appropriate for this combinational calculation. Give outputs defaults so every path defines a value. A missing assignment can infer a latch, an unintended storage element.

> **CHECK YOUR MODEL**
>
> Q4. If a=5, b=2 and input_value=9 before the edge, what are a and b after it?

The ISA defines visible behavior

An **instruction set architecture**, or ISA, defines the operations software can request and the state it can observe. A microarchitecture chooses how to execute those requests.

RV32I is the RISC-V base integer instruction set with 32-bit integer registers. The integer register file has x0 through x31. x0 always reads zero and discards writes. The **program counter**, or PC, identifies the instruction address.

`add x3, x1, x2` means read x1 and x2, add their 32-bit values, and write x3. `addi x1, x0, 7` uses an immediate constant encoded inside the instruction.

RISC-V separates register arithmetic from memory access. `lw x1, 128(x0)` loads a word from byte address 128. `sw x1, 132(x0)` stores a word at byte address 132.

> **SAME CONTRACT, DIFFERENT EXECUTION**
>
> A single-cycle machine and a five-stage pipeline should produce the same architectural instruction results. Their timing and internal state differ.

Instruction	Inputs	Visible result
addi x1, x0, 7	x0 and immediate 7	x1 = 7
add x3, x1, x2	x1 and x2	x3 = x1 + x2
lw x4, 128(x0)	base 0, offset 128	x4 = memory word
sw x4, 132(x0)	address and x4	memory word = x4

> **CHECK YOUR MODEL**
>
> Q5. What happens after addi x0, x0, 9? Which state changes, if any, remain visible?

An instruction word names operands and work

RV32I instructions here are 32 bits long. Decode examines fields within that word. The **opcode** selects a broad instruction class. **funct3** and sometimes **funct7** refine the operation.

In an R-type arithmetic instruction, rd names the destination. rs1 and rs2 name source registers. The bit positions stay fixed across several instruction formats, simplifying decode.

> **WORKED EXAMPLE**
>
> `add x3, x1, x2` encodes as `0x002081B3`. Its opcode is `0x33`, funct3 is 0, funct7 is 0, rd is 3, rs1 is 1 and rs2 is 2.

An I-type instruction uses bits 31:20 for a 12-bit immediate. `addi x1, x0, 7` encodes as `0x00700093`. Ordinary arithmetic immediates are sign-extended. Shift immediates have additional encoding rules.

The local tests construct words with Python encoders. A standard assembler is useful later, but is not required to run these regressions.

FIG 03 / R-TYPE FIELD MAP

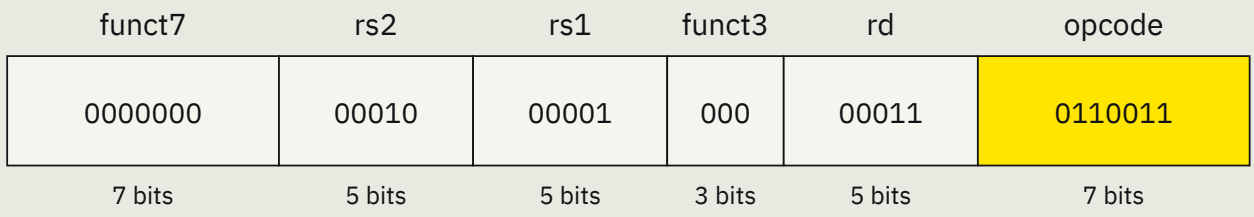


Illustration: field widths sum to 32 bits. Widths use a shared bit scale.

> **CHECK YOUR MODEL**
>
> Q6. What signed value does the 12-bit immediate 0xFFF represent?

One instruction moves through a datapath

A **datapath** carries values through hardware. A control decoder selects the operation, input muxes and permitted side effects. The **ALU**, or arithmetic logic unit, computes integer arithmetic and bitwise results.

For `add x3, x1, x2`, fetch obtains the instruction, decode identifies the registers, the register file supplies operands, and the ALU adds them. A write-enable permits the result to update x3.

A load instead uses the ALU to add the base register and immediate offset. Data memory supplies the final value. A store uses the calculated address plus a separate store-data operand.

A single-cycle implementation completes this path within one clock period. A long path through instruction memory, decode, registers, ALU and data memory can force a long clock.

Control and data must refer to the same instruction. If a pipeline carries a result but loses its destination or write-enable, the value can update the wrong state.

FIG 04 / ADD DATAFLOW

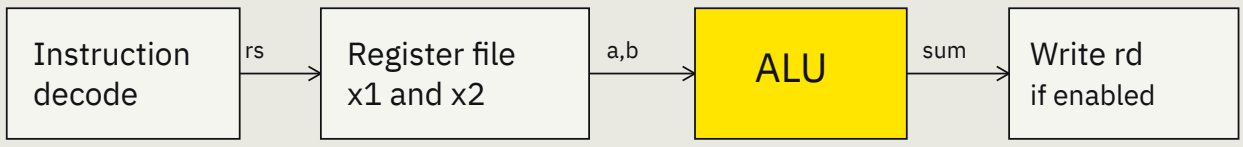


Illustration: values move forward while control selects the permitted update.

Addresses count bytes

A word contains four bytes. In **little-endian** order, the least significant byte sits at the lowest address. Alignment requires a word address divisible by four and a halfword address divisible by two in this core.

> **WORKED EXAMPLE**
>
> Storing `0x80FF007F` at address 128 writes bytes 7F, 00, FF and 80 at addresses 128, 129, 130 and 131.

`lb x1, 131(x0)` returns `0xFFFFFF80`. `lbu x1, 131(x0)` returns `0x00000080`. The memory byte is identical. The load operation selects sign or zero extension.

The core exposes an aligned data word address plus four byte-write strobes. It uses the low effective-address bits to select a byte or halfword. Misaligned halfword and word accesses produce a local trap.

Instruction and data memories are separate. Both reads must be combinational. There is no ready signal or support for waiting on a slow memory.

Byte address	128	129	130	131
Stored byte	0x7F	0x00	0xFF	0x80
Role in word	bits 7:0	bits 15:8	bits 23:16	bits 31:24

> **CHECK YOUR MODEL**
>
> Q7. After that store, what does lhu x2, 130(x0) return?

Different instructions share the machine over time

The five stages are instruction fetch (IF), instruction decode (ID), execute (EX), memory access (MEM) and writeback (WB). Each stage performs different work on a different instruction during the same clock interval.

Registers between stages preserve each instruction's state. A **valid bit** says whether a stage contains an actual instruction. An invalid entry is a **bubble**. A bubble must never update architectural registers or memory.

In the RTL, `f_*` is the IF/ID register and feeds ID. `x_*` feeds EX, `m_*` feeds MEM and `w_*` feeds WB. The name prefix describes stored state, not an additional stage. The fetch PC can be ahead of WB. Each instruction carries its own saved PC.

> **IDEAL TIMING**
>
> For N independent instructions, an ideal five-stage pipeline completes the last WB in `N + 4` cycles, counting initial fill and no stalls. For large N, throughput approaches one retired instruction per cycle.

One instruction still crosses all five stages. Pipeline throughput and instruction latency answer different questions.

FIG 05A / LOGICAL STAGES



Illustration: stage boundaries hold the instruction and its control state.

> **CHECK YOUR MODEL**
>
> Q8. Under those ideal assumptions, how many cycles does a sequence of 10 instructions take?

The pipeline as an indexed conveyor

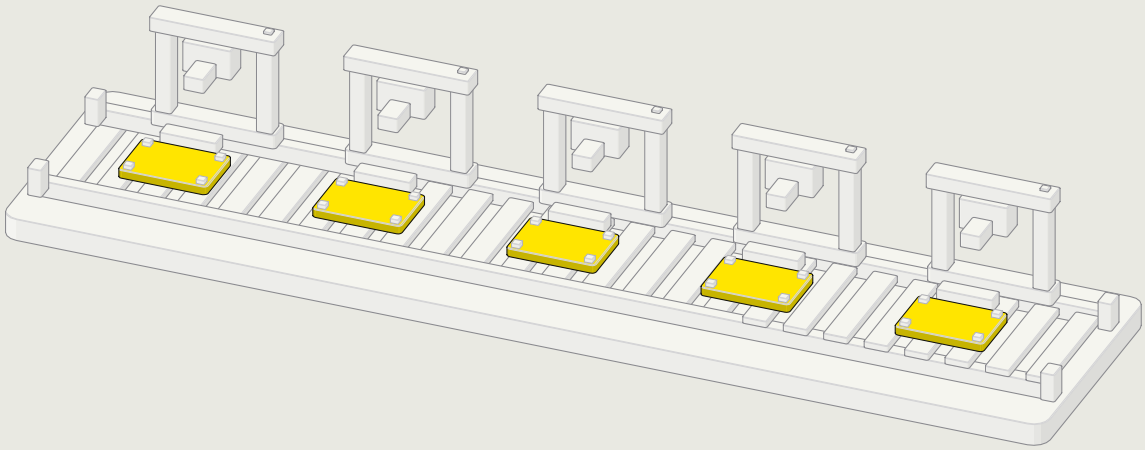
Think of an instruction as a workpiece on a conveyor. Five fixed stations perform different operations. The clock coordinates station updates. A stall can hold younger workpieces while older work continues.

The latches between stations preserve the workpiece's information. The physical model below emphasizes stage occupancy. The timing tables elsewhere show exact dependencies and architectural behavior.

In the HTML edition, select a load-use dependency or a taken branch. The visible workpieces and readout use the same stage-occupancy model.

FIG 05B / PHYSICAL MODEL

Five stations, one clock



IF / fetch	ID / decode	EX / execute	MEM / memory	WB / writeback
I5	I4	I3	I2	I1

Cycle 5. Independent instructions advance one station per clock.

Illustrative physical model. Workpieces represent valid instructions. Exact cycle timing appears in the chapter tables. 106 authored solid parts.

CHECK YOUR MODEL

Use “Next cycle” in the HTML edition. At cycle 5, which instructions occupy IF, ID, EX, MEM and WB? This demonstration uses independent illustrative instructions.

A younger instruction can need an unfinished result

A **data dependency** exists when one instruction uses a result produced by another. A pipeline creates a timing problem if the consumer needs that value before normal writeback supplies it.

```
addi x1, x0, 7
add  x2, x1, x1
```

The ADD needs x1 in EX while the ADDI result has not yet reached WB. Reading the old register file value would give the wrong answer.

Forwarding, also called bypassing, routes an available result directly to the consumer. A **stall** waits when a required value does not exist yet. A **flush** discards work that should no longer execute, such as a wrong-path instruction.

The classic categories are data hazards, control hazards and structural hazards. This design avoids a shared instruction/data memory-port conflict by using separate ports. A single shared port would require arbitration or stalls.

Dependency is about meaning. Hazard handling is about when the machine can deliver the required value.

Mechanism	Purpose	Example
Forward	Use an already computed result	ALU result into next ALU
Stall	Wait for a required value	Immediate load consumer
Flush	Discard younger wrong-path work	Taken branch resolved in EX

Forwarding selects the newest available value

Each EX input can come from its saved ID operand, a WB result, or a newer non-load result in MEM. The hardware compares destination and source register numbers.

```
value = saved_operand
if WB is a valid matching writer: value = WB.result
if MEM is a valid matching non-load writer:
    value = MEM.result
```

The last matching assignment wins in this combinational calculation. MEM represents a younger producer than WB, so its value has priority. x0 never participates as a producer.

> **WORKED EXAMPLE**
>
> If an older instruction writes x1=5 and a newer instruction writes x1=8, a following consumer must use 8. Keeping the WB value when a newer MEM result matches would be wrong.

The same source selection feeds store data, branch comparisons, JALR bases and DOT4. Checking only ordinary ADD operands leaves important hazards unresolved.

FIG 06 / EX OPERAND SELECTION

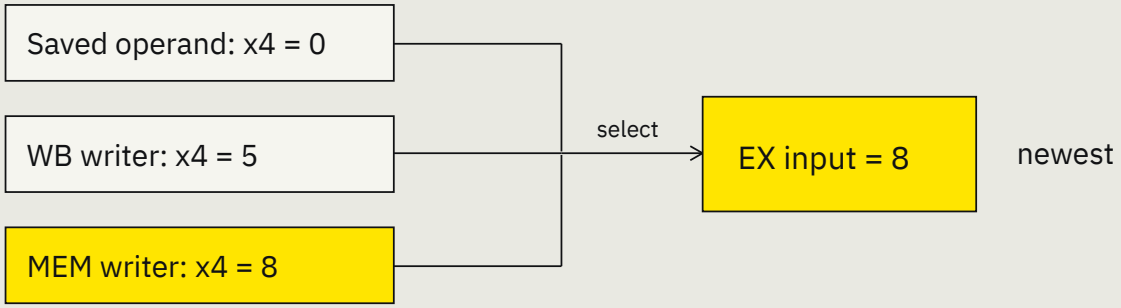


Illustration: the matching non-load MEM result has priority over WB.

> **CHECK YOUR MODEL**
>
> Q9. A consumer matches both a WB writer and a non-load MEM writer for x4. Which wins, and why?

Writeback and decode can meet on one edge

The register file writes at a rising edge. ID/EX also captures its inputs at that edge. With nonblocking assignments, a register-file read expression can still see the old value during evaluation.

```
addi x1, x0, 7
addi x8, x0, 1
addi x9, x0, 2
add  x2, x1, x1
```

When the ADD reaches ID, the first ADDI is in WB. The core explicitly bypasses the WB value into the captured ID operand if the source register matches.

This **WB-to-ID bypass** solves a different timing boundary from the EX operand forwarding muxes. Both can matter in the same implementation.

A good test varies dependency distance. Adjacent instructions exercise one path. Two intervening instructions can exercise another. Also vary whether the dependency appears on rs1, rs2 or both.

For a store, distinguish the address base from the stored value. They are separate dependencies even though both use the same register file.

Clock interval	Producer	Consumer	Needed path
Same interval	WB writes x1	ID captures x1	WB to ID/EX bypass

A load address is not the loaded value

```
lw  x1, 0(x0)
add x2, x1, x1
```

The load calculates its address in EX. Its data becomes available during MEM and is captured into MEM/WB. When the following ADD would enter EX, that loaded value is not yet available through this core's forwarding paths.

The interlock holds PC and IF/ID for one cycle, inserts an invalid EX entry and lets the older load advance. On the next cycle, the ADD enters EX and receives the loaded value from WB.

The hazard detector checks whether the ID instruction really uses rs1 or rs2. Bits in an immediate can occupy the rs2 field position without naming a source register. Treating those bits as a dependency causes unnecessary stalls.

This is a one-cycle stall only under the chosen memory and forwarding contract. A slower memory needs a different protocol and possibly more waiting.

At stall edge	Action
PC	Hold current fetch address
IF/ID	Hold consumer instruction
ID/EX	Insert invalid bubble
EX/MEM and MEM/WB	Older work continues

> **CHECK YOUR MODEL**
>
> Q10. During a load-use stall, which state freezes? Which stage receives the bubble? Does the older load stop?

One bubble preserves the instruction order

The table counts clock intervals. Stage values are visible during each interval and move at its ending edge. “Held” means the instruction stays in its current front-end position.

FIG 07 / LOAD, ADD, STORE

Instruction	1	2	3	4	5	6	7	8
lw x1, 0(x0)	IF	ID	EX	MEM	WB			
add x2, x1, x1		IF	ID	ID held	EX	MEM	WB	
sw x2, 4(x0)			IF	IF held	ID	EX	MEM	WB
EX bubble				bubble				

Illustration: one held front-end cycle creates one invalid EX entry.

At the end of cycle 3, the hazard freezes the front end. The load reaches MEM in cycle 4 while EX holds a bubble. In cycle 5, WB forwards the loaded value to ADD in EX.

In cycle 6, the ADD result in MEM forwards into the store's EX operand selection. The store changes memory at the end of cycle 7 and emits its retirement record after WB in cycle 8.

A memory side effect and the later retirement notification occur at different pipeline points. The checker verifies the ordered byte-write stream as well as retirement records.

> **CHECK YOUR MODEL**
>
> Q11. If the loaded value is 9, what does the ADD produce and what value does the store write?

A taken branch discards two younger stages

```
beq x1, x1, target
sw  x2, 128(x0)      # wrong path
addi x3, x0, 99      # wrong path
target: addi x4, x0, 42
```

The branch decides in EX. At that point, younger instructions occupy ID and IF. A taken branch invalidates both. Fetch resumes at the target on the following cycle.

With no other stalls, this design pays two lost issue opportunities for an EX redirect. An untaken branch continues sequential fetching. Other pipelines can resolve branches later and incur a different penalty.

JAL writes a return address and redirects using a PC-relative immediate. JALR adds a signed immediate to a register base and clears target bit zero. This core traps if the resulting taken target is not word aligned.

Forwarded operands must feed branch comparisons and JALR bases. Otherwise a correct arithmetic pipeline can still choose the wrong instruction path.

FIG 08 / EX-STAGE REDIRECT

Instruction	1	2	3	4	5	6
branch	IF	ID	EX	MEM	WB	
wrong-path store		IF	ID / discard			
wrong-path ADDI			IF / discard			
target				IF	ID	EX

Illustration: the branch discards younger IF and ID instructions at the end of cycle 3.

CHECK YOUR MODEL

Q12. Why is checking only the final x4 value insufficient to prove that the wrong path was squashed?

A trap halts this execution environment

A **trap** reports an exceptional event. Here, ECALL, EBREAK, unsupported instructions and misaligned accesses or taken targets emit a local trap record and halt fetch.

The faulting instruction performs no register or memory side effect. Younger instructions are discarded. Older instructions continue through the remaining stages. Reset starts execution again.

This default build is a small teaching environment. It has no trap vector, privilege state, cause register, interrupt mechanism or recovery handler. The ZICSR=1 build of the same core adds machine-mode trap entry, cause registers and interrupts. Neither build has a memory access-fault response.

FENCE orders memory operations architecturally. The supported base FENCE acts as a no-op under the stated single-hart, ordered, fixed-latency memory contract. FENCE.I is absent, and CSRs exist only in the ZICSR=1 build.

A **Harvard interface** uses separate instruction and data ports. This core additionally assumes combinational reads. FPGA block RAM usually has synchronous reads, so connecting typical block RAM directly changes the timing contract.

An instruction implementation is always interpreted within an execution environment. Documenting that environment prevents a small core from appearing more complete than it is.

DOT4 multiplies four signed byte pairs

The custom instruction treats each source register as four independent signed 8-bit lanes. Lane zero is the least significant byte. It multiplies corresponding lanes and sums all four products into a 32-bit result.

```
DOT4 rd, rs1, rs2
rd = sum(signed8(rs1.byte[i]) * signed8(rs2.byte[i]))
     for i = 0, 1, 2, 3
```

There is no accumulator input, rounding or saturation. The sum ranges from −65024 to 65536. A signed 16-bit result is too narrow. A signed 18-bit value can represent the full range, and the destination remains 32 bits.

The encoding uses custom-0: opcode `0x0B`, funct3=0 and funct7=0. Other function combinations trap. GNU assembly can emit `.insn r 0x0b, 0, 0, a0, a1, a2`.

This project-specific instruction illustrates a small hardware/software tradeoff. It is not a ratified vector extension or evidence of a lab's accelerator architecture.

FIG 09 / DOT4 DATAFLOW

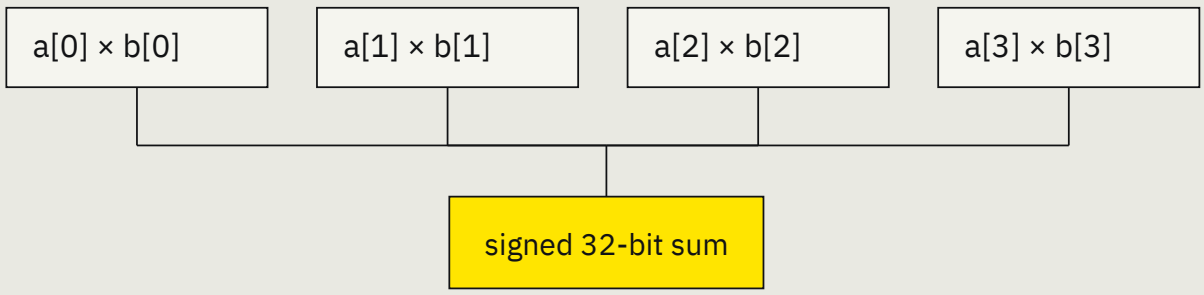


Illustration: independent signed-byte products feed one sum.

Lane interpretation changes the answer

For illustrative source words `A=0x0102FF80` and `B=0x020304FF`, decode each byte as signed before multiplying.

Lane	A byte	A signed	B signed	Product
0	0x80	−128	−1	128
1	0xFF	−1	4	−4
2	0x02	2	3	6
3	0x01	1	2	2
Total				132

The sum is `128 - 4 + 6 + 2 = 132`. The byte 0x80 means −128 here, and 0xFF means −1. Reading the whole word as one signed integer does not implement lane-wise arithmetic.

Four lanes of −128 multiplied by −128 produce `4 × 16384 = 65536`. Four lanes of −128 multiplied by 127 produce `4 × (-16256) = -65024`.

Tests should include isolated lanes, mixed signs, extrema, x0 destinations and dependencies. Directed and random programs cover packed sums. `make dot4-sweep` runs all 262,144 isolated lane pairs through the RTL.

> **CHECK YOUR MODEL**
>
> Q13. DOT4 with A=0x01020304 and B=0x01010101 returns what value? Does DOT4 add the old rd?

The RTL has four useful reading passes

Pass 1: stored state

Find `regs`, `pc` and each stage's valid bit. Follow PC, instruction, destination and write-enable through the pipeline registers.

Pass 2: operand selection and execution

Read the first combinational block for forwarding. Read the opcode case for execution. Notice default control values, legality checks, address generation and the signed DOT4 expression.

Pass 3: hazard and memory boundaries

Find `uses_rs1`, `uses_rs2`, `hazard`, `redirect` and `fault`. Then follow byte strobes, aligned addresses and load extension.

Pass 4: the rising edge

Read reset first. Then follow WB, MEM/WB and EX/MEM updates. Finally inspect the redirect/fault, hazard and ordinary-advance cases. Older stages advance even when the front end stalls.

> **WHERE TO START**
>
> `rtl/rv32_core.sv` contains the core. `tests/tb.sv` supplies clocks and memory. `tests/run.py` supplies programs and expected results. `rtl/timing_top.sv` supplies the implementation measurement shell.

A reference model predicts architectural results

A sequential model executes one instruction at a time without implementing the pipeline. For each instruction, it calculates the next PC, destination register, result and trap behavior.

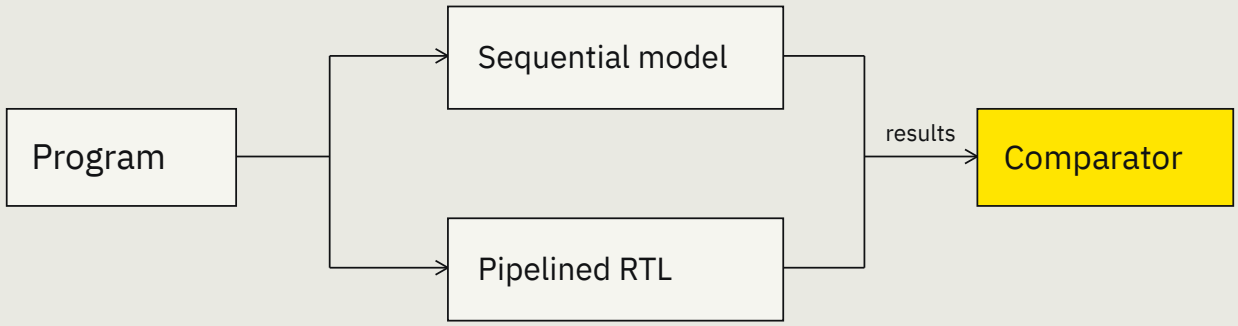
The regression runs the same machine words through that model and the RTL. It compares each retired PC, instruction, destination, value and trap flag. It separately compares every ordered byte write.

This catches errors that a final signature can hide. A wrong-path store might briefly corrupt memory and later be overwritten. A final register check alone can miss it.

The model uses Python integers and explicit 32-bit masking. Signed conversion is deliberate. Python's unlimited integer width does not automatically behave like a 32-bit register.

Both model and RTL belong to this project, so they could share a misreading of the ISA. The core also matches Spike on 39 of 39 riscv-arch-test RV32I tests. ACT4 and formal properties are not integrated.

FIG 10 / TWO EXECUTIONS, ONE COMPARISON



Same program words drive both implementations. A mismatch identifies the first divergent result.

CHECK YOUR MODEL

Q14. Which fields would you inspect first when an instruction retirement differs? Why compare memory writes separately?

Tests should target a reason the core could fail

A **directed test** constructs a known corner case. A **randomized test** explores many combinations using a reproducible seed. Both need an expected result.

For forwarding, cross source operand with producer distance and producer type. Test competing writers, x0 and a source field that is actually an immediate. For loads, cover each size, signedness and aligned byte position.

For control flow, check both outcomes of every branch, JAL, JALR, backward loops and wrong-path side effects. For traps, put a store after the fault and prove it never writes.

An **assertion** checks an invariant during execution. `tests/invariants.svh` checks x0 stability, no writes from invalid stages, front-end stability on a stall and younger-stage squash on a redirect.

> **NEGATIVE EXPERIMENT**
>
> Predict a failing program, deliberately break one forwarding selection in a separate temporary worktree, and observe the first mismatch. Restore the good implementation. Preserve the failure explanation.

The reference adds mid-flight reset tests and a 327-bin hazard cross. Formal integration remains follow-up work. Do not describe it as already checked.

A percentage needs a denominator

Measured reference result: Verilator 5.020 reports 73 of 73 core line instrumentation points hit, or 100.00%, without exclusions. No point is uncovered.

The source is compact, so 73 points are a small denominator. Hitting a line does not test all operands or timing interactions. This metric is narrower than the phrase “100% correct.”

Metric	Reference result	Scope
Passing programs	253 / 253	Directed, random and hazard suite
Retired instructions	11,928 checked	Ordered architectural trace
Line instrumentation	73 / 73 = 100.00%	Core only, Verilator 5.020
Functional bins	46 / 46	Named class/outcome plan
Hazard cross	327 / 327	Producer × consumer × distance

The regression also hits 46 of 46 named functional bins: instruction classes, branch outcomes and trap notification. A separate 327-bin hazard cross covers producer, consumer and distance. The 253 passing programs and 11,928 retired instructions describe the reference run, not every future version.

Adding tests changes counts. Changing source or tool versions can change the line denominator. Reproduce and report your own numbers after edits. Coverage is evidence of exercised behavior, not certification.

> **CHECK YOUR MODEL**
>
> Q15. If all 46 functional bins pass, can a load-to-JALR hazard still be wrong? Explain.

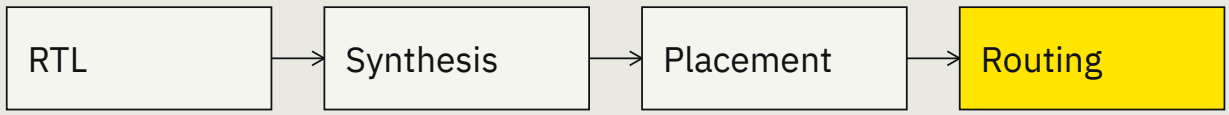
Synthesis maps logic, routing estimates timing

Synthesis turns RTL into a network of target logic resources. A lookup table, or LUT, implements a small Boolean function. Flip-flops store state. Dedicated RAM and DSP blocks can implement larger memories and arithmetic when the device provides them.

Place-and-route assigns resources to physical locations and connects them. Wire delay matters. A synthesis cell count alone cannot establish a routed clock frequency.

The measurement shell registers instruction/load inputs and folds core outputs into an observable register. It keeps the datapath alive during optimization. It contains no program memory, data memory or peripherals.

FIG 11 / IMPLEMENTATION FLOW



The final timing estimate belongs to a device, constraints, tools and route seed.

Measured reference: iCE40 HX8K CT256, requested 12 MHz. Over route seeds 1–5 the internal timing estimate has a median of 36.05 MHz (34.63–37.60). Mapping uses 4,012 LUT4 primitives and 1,692 flip-flops, packed into 5,539 logic cells.

Package I/O is unconstrained. The result is not external timing sign-off or a measured board clock. HX8K provides no hard DSP blocks, so DOT4 uses logic. FPGA resources are not ASIC area in square millimeters.

Clock frequency is only one part of performance

Cycles per instruction, or CPI, divides total cycles by retired instructions. A useful workload also has an instruction count. Execution time is approximately:

time = instruction count × CPI ÷ frequency

> **ILLUSTRATIVE EXAMPLE**
>
> A 100-instruction sequence needs 4 fill cycles, 10 load-use stalls and 16 cycles for eight taken redirects. Total: 130 cycles. CPI: 1.30. At an assumed 20 MHz, that is 6.5 microseconds.

DOT4 can reduce instruction count for packed dot products, but it can also add logic delay. Compare total cycles and each implementation's own clock estimate. Include packing and memory traffic when claiming end-to-end improvement.

RV32I has no integer multiply instruction. A fair scalar RV32I baseline needs a software multiply routine. Compiling a C multiply for RV32IM silently changes the comparison.

The reference benchmark does this. At N=256, DOT4 is 5.2× faster than a table-based RV32I multiply after its clock cost, and 22.2× faster than shift-and-add. Power was not measured.

> **CHECK YOUR MODEL**
>
> Q16. Which is faster for an equal instruction count: 40 MHz at CPI 3, or 20 MHz at CPI 1.5?

A passing program you can explain

Start in the repository. Use a learning branch so your tests and notes have a clear history.

```
git switch -c learning/two-day-core
docker build -t rv32-tools .
docker run --rm --user "$(id -u):$(id -g)" \
  -v "$PWD:/work" rv32-tools make test lint
```

Lab A. Add a named test in `suite()`. Initialize a word, load it into a register, then immediately store that register at a second address. Predict both memory values before running.

Lab B. Take a branch over a store. Predict the retirement path and verify that no byte write occurs at the wrong-path address.

Lab C. Add the mixed-sign DOT4 example from page 21, followed by an instruction that uses its result. Use the local `r(..., op=0x0b)` encoder.

For each lab, write the input, expected result, actual result and explanation into `docs/EXPERIMENT_LOG.md`. Record the command and source revision.

Completion gate: the tests pass, and you can explain which bypass or flush makes each program correct. Passing generated tests alone does not establish your understanding.

A reproducible result with a clear scope

```
docker run --rm --user "$(id -u):$(id -g)" \
  -v "$PWD:/work" rv32-tools \
  make test lint coverage timing
```

Lab D. Open the coverage report. Add an invalid branch-funct3 test and place a store after it. Verify a trap and absence of the younger write. Check whether the missing coverage point becomes hit.

Lab E. Read the route log. Locate the slowest reported internal path. Identify its launch register, logic and destination. Record the device, seed, frequency constraint and estimate.

Lab F. Demonstrate one dependency trace, one DOT4 answer and one measured report. Explain the memory contract and one untested behavior.

Update README numbers from your regenerated reports. Preserve the reference implementation's provenance and describe your specific changes. Commit and push the learning branch. A clean clone should reproduce the documented commands.

Keep the deadline focused. The reference already has Spike checks, a workload benchmark and a board image; reproduce them after the sprint. No physical board run is recorded.

Representation and timing

Q1 0x80 is 128 unsigned. 0xA5 AND 0x0F is 0x05. The mask retains only the lower four bits.

Q2 0xFE is 254−256=−2. Signed 0xFFFFFFFF is −1, so SLT against 1 gives 1. Unsigned 0xFFFFFFFF is 4294967295, so SLTU gives 0.

Q3 $1/(25\times10^{-9}\ \text{s}) = 40$ MHz. This is the period calculation. Actual timing analysis includes register and clock constraints.

Q4 a becomes 9 and b becomes 5. Both nonblocking right-hand sides use the state before the edge.

Q5 x0 remains zero. The instruction executes and sequential control flow advances. There is no register result retained.

Q6 0xFFF is 4095 unsigned. As a signed 12-bit immediate, it is 4095−4096=−1.

Q7 The bytes at 130 and 131 are FF and 80. Little-endian assembly gives 0x80FF. LHU zero-extends it to 0x000080FF.

Q8 10+4=14 cycles under the ideal five-stage fill/drain convention, with no stalls or redirections.

Hazards, arithmetic and evidence

Q9 MEM wins. It holds the newer matching producer. This bypass applies only when its actual non-load result is available and valid.

Q10 PC and IF/ID hold. EX receives an invalid bubble. The older load advances into MEM, then WB.

Q11 The ADD produces 18. The store writes 18 at byte address 4. Store data uses forwarding before capture into EX/MEM.

Q12 x4 can be correct even if the wrong-path store corrupts memory. Check the ordered memory writes and younger retirement records.

Q13 The result is 4+3+2+1=10. DOT4 does not add old rd. It has only two source operands.

Q14 Inspect the first differing PC and instruction, then destination, value and trap flag. Trace earlier producers. Compare memory writes because stores occur before their WB notification.

Q15 Yes. Instruction/outcome bins do not enumerate every producer/consumer timing combination. A dedicated hazard test or stronger properties are still needed.

Q16 They tie under the assumptions: 40/3 and 20/1.5 both give about 13.33 million instructions per second. Work per instruction must also match.

Words to use precisely

Architectural state	Registers, memory effects and control flow visible to software.
Microarchitecture	The hardware organization used to implement an ISA.
Retirement	The point at which an instruction's outcome is reported as completed. This core emits a local WB trace.
Latency	Time for one operation to complete.
Throughput	Completed work per unit time.
Critical path	A timing path that limits the attainable clock under the analysis constraints.
Functional coverage	Hit counts for an explicitly defined behavior plan.
Line coverage	Execution counts for the simulator's source instrumentation points.
Certification	A separate standards-oriented process. A local passing regression is not certification.
Ablation	A controlled experiment that removes a feature to measure its effect.
Evidence	A reproducible observation with its inputs, source, command and limitations.

The repository's source and reports remain the authority for this particular implementation. Instruction semantics come from the ISA specification.

Read the source behind the explanation

This textbook combines official specifications and tools with first-party teaching notes. The diagrams and worked programs are authored illustrations. Measured figures refer to the repository's reference reports.

1. RISC-V ISA manuals: integer semantics, encodings and execution environments.
2. Brown five-stage CPU notes: pipeline pedagogy. Their branch resolution can differ from this core.
3. Verilator simulation guide: coverage modes and interpretation.
4. Yosys documentation and nextpnr: mapping and implementation.
5. Architectural Certification Tests and riscv-formal: the 3.10.0 RV32I tests run against Spike; ACT4 and formal checks are future work.
6. GNU assembler RISC-V formats: custom instruction encoding.
7. Project source and measured reports: actual RTL, test assumptions and implementation artifacts.

See `docs/textbook/SOURCES.md` for chapter-mapped reading. Start the extended build guide after the two-day sprint if you want assertions, matched workload benchmarks or a board demonstration.

The useful outcome is a prediction you can test, a failure you can explain, and a result another person can reproduce.

A short program becomes a regression

Place this entry inside `suite()` in `tests/run.py`. The helpers encode ordinary instruction words. The existing reference model supplies the expected trace and byte writes.

```
yield 'personal_load_store', [
    imm(0, 1, 0, 9),           # addi x1,x0,9
    store(2, 0, 1, 128),       # sw x1,128(x0)
    imm(2, 2, 0, 128, 3),      # lw x2,128(x0)
    store(2, 0, 2, 132),       # sw x2,132(x0)
    0x00100073,                # ebreak
]
```

`imm(funct3, rd, rs1, immediate, opcode)` defaults to opcode 0x13. Passing opcode 3 selects a load. `store(funct3, rs1, rs2, offset)` uses funct3=2 for a word store.

Predict x1=9, x2=9, memory word 128=9 and memory word 132=9. The immediate load-to-store dependency should create one interlock cycle under the current design.

Run `make test` in the tools container. A failure names the test and first different retirement. Inspect `build/personal_load_store.hex` and rerun `vvp build/sim +program=build/personal_load_store.hex` inside that container for the trace.

Keep your handwritten expected values in the experiment log. They help check whether the reference and RTL could share a mistaken interpretation.

The skipped store must never write

This program takes a branch over one store. Branch offsets are relative to the branch instruction's address, in bytes.

```
yield 'personal_wrong_path', [
    imm(0, 1, 0, 7),           # PC 0: x1=7
    branch(0, 1, 1, 8),        # PC 4: beq to 12
    store(2, 0, 1, 128),       # PC 8: discard
    imm(0, 2, 0, 42),          # PC 12: x2=42
    0x00100073,
]
```

The retired PC sequence is 0, 4, 12 and 16. No byte-write event should occur. The branch compares x1 against itself, so it takes the branch regardless of x1's value.

For a DOT4 test, use `r(0, rd, rs1, rs2, op=0x0b)`. Load the desired source words first, then follow DOT4 with a consumer of rd. Check the signed-lane calculation by hand.

For the invalid-branch coverage experiment, encode an unsupported funct3, such as `branch(2, 0, 0, 8)`, and put a store immediately after it. The branch should emit a trap and the younger store should never write.

These recipes are additional learning tests. They are not included in the published 253-program baseline counts. After you add them, regenerate your reports and record the new counts.